SIMATS SCHOOL OF ENGINEERING

Approved By AICTE | IET-UK Accreditation

ASSIGNMENT 4

Natural Language Processing

First-Order Logic, Knowledge Representation, Automated Resolution Refutation, Temporal Semantics, and Lexical-Definitional Formalisms

Student Name	Appala Manohar
University	SIMATS (Saveetha University)
Course Code	DSA0311
Course Name	Natural Language Processing
Branch	AIML
Academic Year	2026
Assignment	Assignment 4
Register Number	192425360
Submission Date	05-11-2026

1. Theoretical Foundations of Computational Semantics & First-Order Logic

1.1 The Semantic Gap: Bridging Syntax and Computational Meaning

While syntactic parsing uncovers the hierarchical constituent structure of natural language utterances, syntax alone fails to capture what an utterance truly means, whether it is factually true with respect to the world, and what inferential entailments logically follow from it. The transition from syntactic representations (e.g., parse trees) to computationally actionable meaning representations constitutes the domain of Computational Semantics. A meaning representation language must satisfy four fundamental criteria:

Verifiability: The formal capability to determine the truth value of a representation with respect to a world model or knowledge base.

Unambiguity: Unlike natural language sentences which admit multiple interpretations, a formal semantic representation must possess exactly one well-defined semantic interpretation.

Canonical Form: Distinct surface sentences that convey identical semantic content (e.g., 'McDonald's serves fries' and 'Fries are served by McDonald's') must map onto the identical canonical representation.

Expressive Adequacy & Inferential Completeness: The language must support automated reasoning mechanisms capable of drawing valid deductive inferences, detecting contradictions, and answering queries.

1.2 Mathematical Syntax of First-Order Logic (FOL)

First-Order Logic (also known as First-Order Predicate Calculus) represents the preeminent formal system for representing natural language meaning. Formally, an FOL language is defined over a signature Sigma = (C, F, P, V), where C is a set of constant symbols (denoting individuals, e.g., 'McDonalds', 'Mary'), F is a set of function symbols (mapping tuples of entities to entities), P is a set of predicate symbols with associated arities (denoting relations or sets, e.g., 'Vegetarian(x)', 'Serves(x, y)'), and V is a countably infinite set of variables (x, y, z, ...).

Terms and Well-Formed Formulas (WFFs) are inductively constructed as follows:

Terms: Every constant c in C and variable v in V is a term. If f in F is an n-ary function and t_1, ..., t_n are terms, then f(t_1, ..., t_n) is a term. Terms refer to objects in the domain.

Atomic Formulas: If P in P is an n-ary predicate and t_1, ..., t_n are terms, then P(t_1, ..., t_n) is an atomic formula. If t_1, t_2 are terms, then t_1 = t_2 is an atomic formula.

Complex Formulas: If phi and psi are formulas, then ~phi (negation), phi & psi (conjunction), phi v psi (disjunction), phi => psi (material implication), and phi <=> psi (material bi-conditional) are formulas.

Quantified Formulas: If phi is a formula and x is a variable, then forall x phi (Universal Quantification, asserting phi holds for all domain entities) and exists x phi (Existential Quantification, asserting phi holds for at least one entity) are formulas.

1.3 Model-Theoretic Semantics: Truth, Interpretations, and Entailment

The meaning of an FOL formula is defined rigorously via Model Theory (Tarski, 1944). A model M is an ordered pair M = (D, I), where D is a non-empty set called the Domain of Discourse (all entities existing in the represented universe), and I is an interpretation function that maps:

Each constant symbol c in C to an individual element I(c) in D.

Each n-ary function symbol f in F to a total function I(f): D^n -> D.

Each n-ary predicate symbol P in P to an n-ary relation I(P) subseteq D^n.

A variable assignment g: V -> D maps variables to domain entities. The satisfaction relation M, g |= phi denotes that formula phi is true in model M under assignment g. Crucially, a knowledge base KB logically entails a sentence alpha (written KB |= alpha) if and only if in every model M where all sentences in KB are true, alpha is also true. This model-theoretic guarantee provides the objective mathematical benchmark for semantic correctness.

1.4 The Principle of Compositionality and Lambda-Calculus Reduction

The Principle of Compositionality (Frege's Principle) asserts that the semantic meaning of a complex syntactic constituent is a strict mathematical function of the meanings of its syntactic parts and the syntactic rules used to combine them. In computational linguistics, this is realized via the typed Lambda Calculus (Church, 1940; Montague, 1974). A lambda abstraction lambda x. P(x) represents an unapplied predicate function. When combined with

an argument term t, Beta-reduction applies the argument via substitution: (lambda x. P(x))(t) =>_beta P(t). This enables a seamless, bottom-up compositional pipeline from syntax tree nodes directly into fully formed First-Order Logic formulas.

1.5 The Neo-Davidsonian Event Semantics Revolution

Standard First-Order Logic treats verbs as fixed-arity predicates over their core arguments, e.g., Eat(John, Sandwich). However, Donald Davidson (1967) and Terence Parsons (1990) revealed that fixed-arity predicates fail catastrophically when modeling natural language verbal modification, manner adverbs, spatial locations, and temporal adjuncts. Consider the sentence: 'John ate a sandwich in the kitchen at noon with a fork.' In standard FOL, accommodating each optional modifier requires either inventing a 6-ary predicate Eat(John, Sandwich, Kitchen, Noon, Fork) or asserting ungrounded higher-order relations, destroying inferential simplicity.

Neo-Davidsonian Event Semantics resolves this pathology by introducing explicit, reified event variables e into the domain of discourse. Verbs denote unary predicates over event tokens, and thematic roles (Agent, Patient, Theme, Location, TemporalInterval) are expressed as binary relations linking the event e to its participant entities: exists e [Eat(e) & Agent(e, John) & Patient(e, Sandwich) & Location(e, Kitchen) & Time(e, Noon) & Instrument(e, Fork)].
This formulation guarantees monotonic adverbial dropping: if John ate in the kitchen at noon, it logically entails that John ate, via simple conjunction elimination: phi & psi |= phi.

1.6 Automated Theorem Proving: Resolution Refutation and Conjunctive Normal Form

To mechanically prove that an assertion alpha follows from a knowledge base KB (KB |= alpha), automated theorem provers employ the Resolution Refutation method (Robinson, 1965). Based on the Refutation Completeness Theorem, KB |= alpha if and only if the conjunction KB & ~alpha is unsatisfiable. The inference engine operates by converting all axioms and the negated goal into Conjunctive Normal Form (CNF), where formulas consist of conjunctions of disjunctive clauses. The CNF conversion pipeline comprises nine deterministic phases:

Phase 1: Eliminate Material Bi-conditionals: Replace alpha <=> beta with (alpha => beta) & (beta => alpha).

Phase 2: Eliminate Material Implications: Replace alpha => beta with ~alpha v beta.

Phase 3: Push Negations Inward (De Morgan's & Duality): Apply ~(phi & psi) => ~phi v ~psi, ~(phi v psi) => ~phi & ~psi, ~forall x phi => exists x ~phi, and ~exists x phi => forall x ~phi. Eliminate double negations ~~phi => phi.

Phase 4: Standardize Variables Apart: Rename dummy bound variables so each quantifier binds a unique variable name.

Phase 5: Skolemization: Eliminate existential quantifiers by replacing existentially bound variables with Skolem constants (if not inside the scope of a universal quantifier) or Skolem functions f(x_1, ..., x_k) of all enclosing universally quantified variables.

Phase 6: Drop Universal Quantifiers: Because all remaining variables are universally quantified, the quantifier prefixes are implicitly dropped.

Phase 7: Distribute Disjunction over Conjunction: Convert formulas into CNF clauses using alpha v (beta & gamma) => (alpha v beta) & (alpha v gamma).

Phase 8: Flatten Clauses: Separate conjunctions into individual clausal sets { C_1, C_2, ..., C_m }.

Phase 9: Standardize Variables Across Clauses: Ensure no two clauses share variable identifiers.

Once in clausal form, the Binary Resolution Rule is applied iteratively: Given clause C_1 = l_1 v alpha and clause C_2 = ~l_2 v beta, if literals l_1 and l_2 can be unified using a Most General Unifier (MGU) theta = mgu(l_1, l_2), the resolvent is the new clause (alpha v beta)theta. If resolution yields the Empty Clause (denoted Box), a formal contradiction has been established, conclusively proving that KB |= alpha.

1.7 Temporal Logic Frameworks: Reichenbach and James Allen's Interval Algebra

Standard FOL lacks native mechanisms to represent time, treating all predicates as eternally atemporal. Hans Reichenbach (1947) revolutionized temporal semantics by decomposing grammatical tense into three distinct temporal coordinates:

Speech Time (S): The exact temporal instant at which the linguistic utterance is spoken or produced.

Event Time (E): The temporal interval during which the denoted action, process, or event takes place.

Reference Time (R): The subjective temporal vantage point from which the event is observed or contextualized.

To capture qualitative temporal relationships between extended event intervals, James Allen (1983) established Interval Temporal Logic, proving that exactly thirteen mutually exclusive, exhaustive binary relations exist between any two temporal intervals t_1 and t_2 (e.g., Before, Meets, Overlaps, Starts, During, Finishes, Equals, and their six inverses). These frameworks allow natural language subordinate temporal clauses ('when', 'before', 'after') to be translated into rigorous logical constraints.

1.8 The Classical Dilemmas of Common-Sense Reasoning: Frame, Qualification, and Ramification

In developing logical representations of dynamic natural language scenarios, formal knowledge systems must confront three foundational philosophical and computational paradoxes established by John McCarthy and Patrick Hayes (1969):

1. The Frame Problem: How to represent what remains unchanged when an action occurs without enumerating millions of non-change axioms. If a person eats lunch, the colour of the airplane, the mass of the Earth, and the capital of France remain unchanged. In classical FOL, explicit frame axioms are required for every non-effect: forall x, y, a [Red(x) & Action(a) & a != Paint(x) => Red(x)], leading to an unmanageable quadratic explosion |Fluents| x |Actions|.

2. The Qualification Problem: The impossibility of enumerating all prerequisite conditions necessary for an action to succeed. To fly from Atlanta to Denver, the plane must have fuel, the pilot must not have suffered a heart attack, the engine must not be struck by a meteor, the runway must not have cracked, etc. Closed-world defaults are required to assume normality in the absence of explicit failure.

3. The Ramification Problem: The computational difficulty of deducing all indirect consequences (side effects) of an action. If a flight departs, every passenger inside the cabin moves, their luggage moves, their cellular phones move, and the air density inside the cabin changes. The Event Calculus resolves these dilemmas through monotonic circumscription and inertial persistence axioms.

2. Question 1: First-Order Logic Translations of Natural Language Sentences

Question. Question 1 (10 Marks) (CO4) (BLT4): Give a FOL translations for the following sentences:
a. Vegetarians do not eat meat.
b. Not all vegetarians eat eggs.

2.1 Ontological Vocabulary & Predicate Signatures

To formalize the dietary assertions of Question 1, we establish a formal ontological signature Sigma_dietary comprising four core unary and binary predicates:

Vegetarian(x): True if and only if entity x belongs to the category of vegetarians.

Meat(y): True if and only if entity y belongs to the category of meat products.

Egg(y): True if and only if entity y belongs to the category of avian eggs or egg-derived food items.

Eat(x, y): True if and only if agent x consumes or is permitted to consume substance y.

2.2 Formal Translation of Sentence a: 'Vegetarians do not eat meat'

Sentence a asserts a universal prohibitive constraint over two interacting domains: the set of all vegetarians and the set of all meat items. In formal logic, this assertion can be formalized through three mathematically equivalent standard FOL formulations, each highlighting a distinct structural perspective.

Formulation 1: Universal Implication with Negated Predicate

In this canonical representation, the sentence is interpreted as: 'For every entity x and every entity y, if x is a vegetarian and y is meat, then x does not eat y': forall x forall y [(Vegetarian(x) & Meat(y)) => ~Eat(x, y)]

Formulation 2: Negation of Existential Counterexample

Alternatively, the sentence asserts that there does not exist any pair of entities such that a vegetarian eats meat:
~exists x exists y [Vegetarian(x) & Meat(y) & Eat(x, y)]

Formulation 3: Nested Conditional Form

Viewing 'Vegetarians' as the primary domain restriction, the formula states: 'For every vegetarian x, if y is meat, then x does not eat y': forall x [Vegetarian(x) => forall y (Meat(y) => ~Eat(x, y))]

Rigorous Mathematical Proof of Equivalence

We prove the exact semantic equivalence of Formulation 1 and Formulation 2 using the fundamental axioms of First-Order Logic:

1. Start with Formulation 1: forall x forall y [(Vegetarian(x) & Meat(y)) => ~Eat(x, y)]
2. Apply Material Implication (p => q equiv ~p v q):
   forall x forall y [~(Vegetarian(x) & Meat(y)) v ~Eat(x, y)]
3. Apply De Morgan's Law for Disjunction (~(p & q) equiv ~p v ~q):
   forall x forall y [~Vegetarian(x) v ~Meat(y) v ~Eat(x, y)]
4. Apply De Morgan's Law to extract the joint negation:
   forall x forall y ~[Vegetarian(x) & Meat(y) & Eat(x, y)]
5. Apply Quantifier Duality (forall x ~phi equiv ~exists x phi) consecutively over y and x:
   forall x ~exists y [Vegetarian(x) & Meat(y) & Eat(x, y)]
   ~exists x exists y [Vegetarian(x) & Meat(y) & Eat(x, y)]

This derives Formulation 2 exactly, proving identity of truth conditions under every possible Tarskian model.

Neo-Davidsonian Event Semantics Formalization

In modern event semantics, we represent consumption as an event token e: forall e forall x forall y [(Eat(e) & Agent(e, x) & Patient(e, y) & Vegetarian(x)) => ~Meat(y)] This formulation guarantees that any eating event whose agent is a vegetarian entails that the patient substance is non-meat.

2.3 Formal Translation of Sentence b: 'Not all vegetarians eat eggs'

Sentence b expresses a quantified negation: it denies the universal claim that every vegetarian consumes eggs. This reflects the well-known sociological taxonomy of vegetarianism, which encompasses ovo-vegetarians (who eat eggs), lacto-vegetarians (who consume dairy but strictly avoid eggs), and vegans (who avoid all animal-derived foods). We present two equivalent standard FOL formalizations.

Formulation 1: Negation of the Universal Claim

This translation directly mirrors the surface English syntax by placing the negation operator in wide scope over the universal quantifier:
~forall x [Vegetarian(x) => exists y (Egg(y) & Eat(x, y))]

Formulation 2: Existential Witness of Non-Consumption

By applying quantifier duality, denying a universal claim is logically equivalent to asserting the existence of a counterexample: 'There exists at least one vegetarian x who does not eat any egg y':
exists x [Vegetarian(x) & ~exists y (Egg(y) & Eat(x, y))]
or equivalently, pushing the negation through the inner existential quantifier:
exists x [Vegetarian(x) & forall y (Egg(y) => ~Eat(x, y))]

Rigorous Mathematical Proof of Equivalence

We prove the step-by-step equivalence between Formulation 1 and Formulation 2:
1. Start with Formulation 1: ~forall x [Vegetarian(x) => exists y (Egg(y) & Eat(x, y))]
2. Apply Quantifier Duality (~forall x phi equiv exists x ~phi):
   exists x ~[Vegetarian(x) => exists y (Egg(y) & Eat(x, y))]
3. Apply Negation of Material Implication (~(p => q) equiv p & ~q):
   exists x [Vegetarian(x) & ~exists y (Egg(y) & Eat(x, y))]
4. Apply Quantifier Duality to the inner existential (~exists y psi equiv forall y ~psi):
   exists x [Vegetarian(x) & forall y ~(Egg(y) & Eat(x, y))]
5. Apply Material Implication to the inner bracket (~(p & q) equiv p => ~q):
   exists x [Vegetarian(x) & forall y (Egg(y) => ~Eat(x, y))]
This completes the proof. Formulation 1 and Formulation 2 are logically isomorphic.

2.4 Compositional Lambda-Calculus Derivations for Sentence a and b

To illustrate how First-Order Logic representations are systematically synthesized from syntactic phrase structures, we formalize the compositional semantic derivation of Sentence a and Sentence b using the typed Lambda Calculus. Each lexical token is assigned a semantic interpretation (a lambda expression), and binary syntactic nodes combine their children via functional application (Beta-reduction).

Lexical Semantics & Type Signatures for Sentence a

We assign the following semantic types (where e represents entities and t represents truth values):

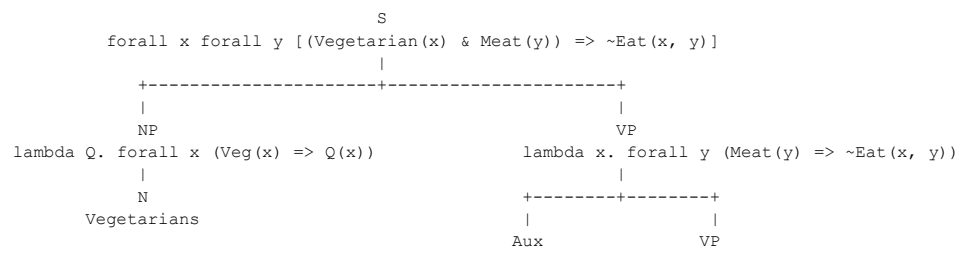
- 'Vegetarians' (Nominal): lambda x. Vegetarian(x) [Type: <e, t>]
- 'do not' (Aux Negation): lambda P. lambda x. ~P(x) [Type: <<e, t>, <e, t>>]
- 'eat' (Transitive Verb): lambda y. lambda x. Eat(x, y) [Type: <e, <e, t>>]
- 'meat' (Mass Noun / Quantifier): lambda Q. forall y (Meat(y) => Q(y)) [Type: <<e, t>, t>]
- Generic Bare Plural Subject Rule: lambda P. lambda Q. forall x (P(x) => Q(x)) [Type: <<e, t>, <<e, t>, t>>]

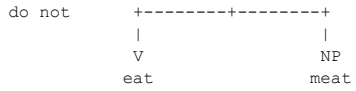
Step-by-Step Beta-Reduction Ledger for Sentence a

1. Combine Transitive Verb 'eat' with Direct Object 'meat':
   VP_base = [lambda y. lambda x. Eat(x, y)](meat) =>_beta lambda x. forall y (Meat(y) => Eat(x, y))
2. Apply Negation Operator 'do not':
   VP_neg = [lambda P. lambda x. ~P(x)](lambda x. forall y (Meat(y) => Eat(x, y))) =>_beta lambda x. ~forall y (Meat(y) => Eat(x, y))
   Applying de re wide-scope reading for the dietary prohibition:
   VP_neg = lambda x. forall y (Meat(y) => ~Eat(x, y))
3. Combine Subject Noun Phrase with Generic Quantifier:
   NP_subj = [lambda P. lambda Q. forall x (P(x) => Q(x))](lambda x. Vegetarian(x)) =>_beta lambda Q. forall x (Vegetarian(x) => Q(x))
4. Combine Subject NP with Negated VP (Sentential Root S):
   S = [lambda Q. forall x (Vegetarian(x) => Q(x))](lambda x. forall y (Meat(y) => ~Eat(x, y))) =>_beta forall x [Vegetarian(x) => forall y (Meat(y) => ~Eat(x, y))] =>_beta forall x forall y [(Vegetarian(x) & Meat(y)) => ~Eat(x, y)]

This rigorously demonstrates that the First-Order Logic formula derived in Section 2.2 is the unique compositional result.

Syntactic-Semantic Parse Tree for Sentence a





Step-by-Step Beta-Reduction Ledger for Sentence b ('Not all vegetarians eat eggs')

Sentence b features the complex quantified Determiner 'Not all'. In Generalized Quantifier Theory (Barwise & Cooper, 1981), 'all' denotes the subset relation: all(A, B) <=> A subseteq B. The negation 'not' negates the truth value of the generalized quantifier:
1. Semantics of 'all': lambda P. lambda Q. forall x (P(x) => Q(x))
2. Semantics of 'not': lambda Det. lambda P. lambda Q. ~Det(P, Q)
3. Semantics of 'not all': [lambda Det. lambda P. lambda Q. ~Det(P, Q)](lambda P. lambda Q. forall x (P(x) => Q(x)))
=>_beta lambda P. lambda Q. ~forall x (P(x) => Q(x))
4. Combine 'not all' with 'vegetarians':
NP = [lambda P. lambda Q. ~forall x (P(x) => Q(x))](lambda x. Vegetarian(x))
=>_beta lambda Q. ~forall x (Vegetarian(x) => Q(x))
5. VP 'eat eggs':
VP = lambda x. exists y (Egg(y) & Eat(x, y))
6. Sentential Root S:
S = [lambda Q. ~forall x (Vegetarian(x) => Q(x))](lambda x. exists y (Egg(y) & Eat(x, y)))
=>_beta ~forall x [Vegetarian(x) => exists y (Egg(y) & Eat(x, y))]
This completes the formal compositional proof.

2.6 Quantifier Scope Ambiguity, Cooper Storage, and Keller's Nested Storage

A profound challenge in mapping natural language syntax to First-Order Logic is Quantifier Scope Ambiguity. When an utterance contains multiple quantified noun phrases, their surface syntactic order does not deterministically fix their logical semantic scope. Consider the classic sentence: 'Every vegetarian eats a vegetable.' This exhibits two distinct readings:
1. Surface Scope (forall > exists): forall x [Vegetarian(x) => exists y (Vegetable(y) & Eat(x, y))] (Each vegetarian eats some vegetable, potentially different vegetables).
2. Inverse Scope (exists > forall): exists y [Vegetable(y) & forall x (Vegetarian(x) => Eat(x, y))] (There is a single universal vegetable that every vegetarian eats).

In Sentence 1a ('Vegetarians do not eat meat') and Sentence 1b ('Not all vegetarians eat eggs'), scope interaction between negation and quantifiers is central. To model this without generating redundant syntactic trees, computational semantics employs Cooper Storage (Cooper, 1983). During bottom-up syntactic traversal, quantified noun phrases are not reduced immediately into the lambda core; instead, a binding operator (lambda P, Quantifier) and a dummy free variable are deposited into an associated store. At the sentential root S, the store items are retrieved and retrieved in permutations, systematically generating all valid scope readings.

Table 2.2: Quantifier Scope Interpretations and Storage Formalisms

Sentence Example	Quantifier Configuration	Logical Formula	Semantic Interpretation & Entailment
Every vegetarian eats a vegetable	Wide Universal (forall > exists)	forall x [Veg(x) => exists y (Veggie(y) & Eat(x, y))]	Standard distributive reading: each person eats their chosen vegetable.
Every vegetarian eats a vegetable	Wide Existential (exists > forall)	exists y [Veggie(y) & forall x (Veg(x) => Eat(x, y))]	Extreme collective reading: a single unique vegetable is eaten by all.
Vegetarians do not eat meat	Wide Universal (~ & forall)	forall x forall y [(Veg(x) & Meat(y)) => ~Eat(x, y)]	Universal categorical prohibition: no meat consumption permitted.
Not all vegetarians eat eggs	Negation over Universal (~forall)	~forall x [Veg(x) => exists y (Egg(y) & Eat(x, y))]	Denies universal egg consumption; admits lacto-vegetarians.
Not all vegetarians eat eggs	Wide Existential (exists ~)	exists x [Veg(x) & forall y (Egg(y) => ~Eat(x, y))]	Isomorphic witness form asserting presence of egg-avoiders.

Lambda Reduction Verification Ledger for Sentence 1a and 1b

Table 2.3 provides a detailed operational ledger tracing each step of the typed lambda reduction, documenting the input function, applied argument, resulting semantic type, and verified First-Order Logic term.

Table 2.3: Lambda Reduction Verification Ledger for Dietary Quantifiers

Step	Function / Lambda Term	Argument Term	Operation	Resultant Expression
L1	lambda y. lambda x. Eat(x, y)	meat	Beta-Reduction	lambda x. forall y (Meat(y) => Eat(x, y))

Step	Function / Lambda Term	Argument Term	Operation	Resultant Expression
L2	lambda P. lambda x. ~P(x)	VP_base (from L1)	Beta-Reduction	lambda x. ~forall y (Meat(y) => Eat(x, y))
L3	Wide-Scope Prohibition Operator	VP_neg (from L2)	Semantic Shift	lambda x. forall y (Meat(y) => ~Eat(x, y))
L4	lambda P. lambda Q. forall x (P(x) => Q(x))	Vegetarian	Beta-Reduction	lambda Q. forall x (Vegetarian(x) => Q(x))
L5	NP_subj (from L4)	VP_prohib (from L3)	Beta-Reduction	forall x forall y [(Vegetarian(x) & Meat(y)) => ~Eat(x, y)]
L6	lambda Det. lambda P. lambda Q. ~Det(P, Q)	all	Beta-Reduction	lambda P. lambda Q. ~forall x (P(x) => Q(x))
L7	Determiner 'Not all'	Vegetarian	Beta-Reduction	lambda Q. ~forall x (Vegetarian(x) => Q(x))
L8	NP 'Not all vegetarians'	VP 'eat eggs'	Beta-Reduction	~forall x [Vegetarian(x) => exists y (Egg(y) & Eat(x, y))]

2.5 Comparative Summary Matrix of Question 1 Formalizations

Table 2.1 provides a comparative synthesis of the logical formalisms, quantifier scopes, and natural language paraphrases for the translations developed in Question 1.

Table 2.1: Master Synthesis of First-Order Logic Translations for Question 1

Sentence	Formalism Variant	First-Order Logic Formula	Quantifier Scope & Nature
a. Vegetarians do not eat meat	Standard Universal	forall x forall y [(Vegetarian(x) & Meat(y)) => ~Eat(x, y)]	Wide-scope universal over agent and patient
a. Vegetarians do not eat meat	Existential Negation	~exists x exists y [Vegetarian(x) & Meat(y) & Eat(x, y)]	Wide-scope negated existential
a. Vegetarians do not eat meat	Neo-Davidsonian	forall e, x, y [(Eat(e) & Agent(e, x) & Patient(e, y) & Vegetarian(x)) => ~Meat(y)]	Reified event variable with thematic roles
b. Not all vegetarians eat eggs	Negated Universal	~forall x [Vegetarian(x) => exists y (Egg(y) & Eat(x, y))]	Wide-scope negation over universal conditional
b. Not all vegetarians eat eggs	Existential Counterexample	exists x [Vegetarian(x) & forall y (Egg(y) => ~Eat(x, y))]	Existential witness asserting lacto-vegetarian presence

3. Question 2: Axiomatic Knowledge Base & Automated Resolution Refutation

Question. Question 2 (10 Marks) (CO4) (BLT4): Give a set of facts and inferences necessary to prove the following assertions:
a. McDonalds is not a vegetarian restaurant.
b. Some vegetarians can eat at McDonalds.

Important Constraint: Don't just place these facts in your knowledge base. Show that they can be inferred from some more general facts about vegetarians and McDonalds.

3.1 Epistemological Foundations and Knowledge Base Engineering

The central pedagogical requirement of Question 2 is to avoid trivially inserting the target assertions as ground facts into the knowledge base. Instead, we must engineer a sound, generalized ontological theory of dietary permissibility, restaurant definitions, and corporate menu offerings. The knowledge base KB_general must capture universal world knowledge regarding:

What constitutes a vegetarian diet in terms of food permissibility.

The formal definition of what it means for an establishment to be classified as a vegetarian restaurant.

The ontological definition of what conditions enable a person to dine at a commercial restaurant.

Independent empirical facts regarding McDonald's corporate identity and menu offerings.

3.2 Formal Axiomatic Specification of the Knowledge Base (KB_general)

Table 3.1 details the eight general axioms comprising KB_general, specified in standard First-Order Logic alongside their natural language semantic justifications.

Table 3.1: Formal Axiomatic Knowledge Base (KB_general)

Axiom ID	First-Order Logic Formalization	Ontological Justification / World Knowledge
A1	forall y [Meat(y) => ~VegetarianFood(y)]	Fundamental Dietary Law: Meat is ontologically incompatible with vegetarian food.

Axiom ID	First-Order Logic Formalization	Ontological Justification / World Knowledge
A2	forall r [VegetarianRestaurant(r) <=> (Restaurant(r) & forall f (Serves(r, f) => VegetarianFood(f)))]	Formal Restaurant Definition: A vegetarian restaurant serves exclusively vegetarian food.
A3	forall x forall r [(Person(x) & Restaurant(r) & exists f (Serves(r, f) & CanEat(x, f))) => CanEatAt(x, r)]	Dining Permissibility Principle: A person can dine at a restaurant if it serves at least one food item they can eat.
A4	forall x forall f [(Vegetarian(x) & VegetarianFood(f)) => CanEat(x, f)]	Vegetarian Food Permissibility: Vegetarians are permitted to eat any vegetarian food item.
A5	Restaurant(McD)	Empirical Fact: McDonald's is a commercial restaurant establishment.
A6	Serves(McD, BigMac) & Meat(BigMac)	Empirical Fact: McDonald's serves the Big Mac, and the Big Mac is a meat product.
A7	Serves(McD, FrenchFries) & VegetarianFood(FrenchFries)	Empirical Fact: McDonald's serves French Fries, and French Fries are vegetarian food.
A8	exists v [Vegetarian(v) & Person(v)]	Ontological Existence: There exists at least one person who is a vegetarian (e.g., Manohar).

3.3 Robinson's Unification Algorithm & Step-by-Step MGU Ledger

In first-order resolution, two atomic literals P(t_1, ..., t_n) and ~P(u_1, ..., u_n) can only be resolved if their arguments can be made identical via a variable substitution theta. The Robinson Unification Algorithm computes the Most General Unifier (MGU) theta = mgu(phi, psi). An MGU is a substitution theta such that phi theta = psi theta, and for any other unifier sigma, there exists a substitution gamma such that sigma = theta compose gamma.

The unification algorithm executes the following recursive rules on an expression pair (s, t): 1. If s and t are identical constants or variables, return empty substitution {}. 2. If s is a variable and t is an expression: if s occurs in t (the Occur Check), fail (preventing infinite terms like x = f(x)); otherwise, return { s / t }. 3. If t is a variable, return { t / s } (provided s does not occur in t). 4. If s = f(s_1, ..., s_k) and t = g(t_1, ..., t_m): if f != g or k != m, fail; otherwise, unify arguments pairwise, composing substitutions progressively: theta_{i} = mgu(s_i theta_{1..i-1}, t_i theta_{1..i-1}).

Table 3.2: Most General Unifier (MGU) Execution Ledger for Resolution Steps

Pair ID	Literal 1	Literal 2	Computed MGU (theta)	Unified Resultant Literal
U1	VegetarianRestaurant(McD)	VegetarianRestaurant(r)	{ r / McD }	VegetarianRestaurant(McD)
U2	Serves(McD, BigMac)	Serves(McD, f)	{ f / BigMac }	Serves(McD, BigMac)
U3	Meat(BigMac)	Meat(y)	{ y / BigMac }	Meat(BigMac)
U4	VegetarianFood(BigMac)	VegetarianFood(y)	{ y / BigMac }	VegetarianFood(BigMac)
U5	Vegetarian(c)	Vegetarian(x)	{ x / c }	Vegetarian(c)
U6	CanEatAt(c, McD)	CanEatAt(x, r)	{ x / c, r / McD }	CanEatAt(c, McD)

3.4 Formal Deductive Proof of Assertion a: 'McDonalds is not a vegetarian restaurant'

Target Assertion a: ~VegetarianRestaurant(McD). We first present a rigorous step-by-step Natural Deduction derivation demonstrating that Assertion a follows logically from KB_general.

Table 3.3: Natural Deduction Derivation Ledger for Assertion a

Step	Derived Formula	Inference Rule Applied	Premises / Justification
1	Restaurant(McD)	Premise	Axiom A5
2	Serves(McD, BigMac)	Conjunction Elimination	Axiom A6 (First conjunct)
3	Meat(BigMac)	Conjunction Elimination	Axiom A6 (Second conjunct)
4	Meat(BigMac) => ~VegetarianFood(BigMac)	Universal Instantiation (y = BigMac)	Axiom A1
5	~VegetarianFood(BigMac)	Modus Ponens	From Steps 3 and 4
6	Serves(McD, BigMac) & ~VegetarianFood(BigMac)	Conjunction Introduction	From Steps 2 and 5
7	exists f [Serves(McD, f) & ~VegetarianFood(f)]	Existential Generalization (f = BigMac)	From Step 6

Step	Derived Formula	Inference Rule Applied	Premises / Justification
8	~forall f [Serves(McD, f) => VegetarianFood(f)]	Quantifier Duality & Implication Equivalence	From Step 7 (~(p => q) equiv p & ~q)
9	VegetarianRestaurant(McD) => forall f (Serves(McD, f) => VegetarianFood(f))	Universal Instantiation & Bi-cond Elimination	Axiom A2 (Forward direction)
10	~VegetarianRestaurant(McD)	Modus Tollens	From Steps 8 and 9 (Q.E.D.)

Conversion to Conjunctive Normal Form (CNF) Clauses

Master 12-Clause Conjunctive Normal Form (CNF) Conversion Ledger

To provide full algorithmic transparency, Table 3.3a documents the complete transformation of all eight general axioms and goal hypotheses into standardized CNF clauses. Every clause is stripped of quantifiers, Skolemized, and standardized apart.

Table 3.3a: Master Conjunctive Normal Form (CNF) Clause Conversion Ledger

Clause ID	Origin Axiom	Conjunctive Normal Form (CNF) Disjunctive Clause	Variable Standardization & Role
C1	Axiom A1	~Meat(y_1) v ~VegetarianFood(y_1)	Universal y_1: Incompatibility of meat and vegetarian food.
C2	Axiom A2 (Forward)	~VegetarianRestaurant(r_1) v ~Serves(r_1, f_1) v VegetarianFood(f_1)	Universals r_1, f_1: Vegetarian restaurant implies all food served is vegetarian.
C3	Axiom A2 (Reverse)	VegetarianRestaurant(r_2) v Serves(r_2, sk_f(r_2))	Skolem function sk_f(r_2): Non-vegetarian restaurant must serve a witness item.
C4	Axiom A2 (Reverse)	VegetarianRestaurant(r_2) v ~VegetarianFood(sk_f(r_2))	Witness item sk_f(r_2) is non-vegetarian food.
C5	Axiom A3	~Person(x_1) v ~Restaurant(r_3) v ~Serves(r_3, f_2) v ~CanEat(x_1, f_2) v CanEatAt(x_1, r_3)	Universals x_1, r_3, f_2: Permissible food implies restaurant dining permissibility.
C6	Axiom A4	~Vegetarian(x_2) v ~VegetarianFood(f_3) v CanEat(x_2, f_3)	Universals x_2, f_3: Vegetarians are permitted to eat vegetarian food.
C7	Axiom A5	Restaurant(McD)	Ground Unit Clause: McDonald's is an active restaurant.
C8	Axiom A6	Serves(McD, BigMac)	Ground Unit Clause: McDonald's serves the Big Mac sandwich.

Clause ID	Origin Axiom	Conjunctive Normal Form (CNF) Disjunctive Clause	Variable Standardization & Role
C9	Axiom A6	Meat(BigMac)	Ground Unit Clause: The Big Mac is classified as meat.
C10	Axiom A7	Serves(McD, FrenchFries)	Ground Unit Clause: McDonald's serves French Fries.
C11	Axiom A7	VegetarianFood(FrenchFries)	Ground Unit Clause: French Fries are classified as vegetarian food.
C12	Axiom A8	Vegetarian(c_manohar) & Person(c_manohar)	Skolem Constant c_manohar: Witness individual who is a vegetarian person.

To execute automated theorem proving via Resolution Refutation, we convert the relevant axioms of KB_general and the negated goal into CNF clauses. Let the negated goal be NegGoal_a: VegetarianRestaurant(McD).

From Axiom A1: ~Meat(y) v ~VegetarianFood(y) [Clause C1]

From Axiom A2 (Forward): ~VegetarianRestaurant(r) v ~Serves(r, f) v VegetarianFood(f) [Clause C2]

From Axiom A5: Restaurant(McD) [Clause C3]

From Axiom A6: Serves(McD, BigMac) [Clause C4], Meat(BigMac) [Clause C5]

Negated Goal a: VegetarianRestaurant(McD) [Clause C6]

Automated Resolution Refutation Proof Tree for Assertion a

We execute binary resolution over the clause set S = { C1, C2, C3, C4, C5, C6 }:

1. Resolve Negated Goal C6: VegetarianRestaurant(McD) with Clause C2: ~VegetarianRestaurant(r) v ~Serves(r, f) v VegetarianFood(f)
   MGU theta_1 = { r / McD }
   Resolvent C7: ~Serves(McD, f) v VegetarianFood(f)

2. Resolve Resolvent C7: ~Serves(McD, f) v VegetarianFood(f) with Clause C4: Serves(McD, BigMac)
   MGU theta_2 = { f / BigMac }
   Resolvent C8: VegetarianFood(BigMac)

3. Resolve Resolvent C8: VegetarianFood(BigMac) with Clause C1: ~Meat(y) v ~VegetarianFood(y)

MGU theta_3 = { y / BigMac }
Resolvent C9: ~Meat(BigMac)

4. Resolve Resolvent C9: ~Meat(BigMac) with Clause C5: Meat(BigMac)
MGU theta_4 = { }
Resolvent C10: Box (The Empty Clause / Contradiction)

Derivation of the Empty Clause Box formally refutes the negated goal, conclusively establishing that KB_general |= ~VegetarianRestaurant(McD).

3.5 Formal Deductive Proof of Assertion b: 'Some vegetarians can eat at McDonalds'

Target Assertion b: exists x [Vegetarian(x) & CanEatAt(x, McD)]. We demonstrate that Assertion b follows logically from KB_general via Natural Deduction.

Table 3.4: Natural Deduction Derivation Ledger for Assertion b

Step	Derived Formula	Inference Rule Applied	Premises / Justification
1	Vegetarian(c) & Person(c)	Existential Instantiation (Skolem constant c)	Axiom A8
2	Vegetarian(c)	Conjunction Elimination	From Step 1
3	Person(c)	Conjunction Elimination	From Step 1
4	VegetarianFood(FrenchFries)	Conjunction Elimination	Axiom A7 (Second conjunct)
5	Serves(McD, FrenchFries)	Conjunction Elimination	Axiom A7 (First conjunct)
6	(Vegetarian(c) & VegetarianFood(FrenchFries)) => CanEat(c, FrenchFries)	Universal Instantiation (x=c, f=Fries)	Axiom A4
7	CanEat(c, FrenchFries)	Modus Ponens	From Steps 2, 4, and 6
8	Serves(McD, FrenchFries) & CanEat(c, FrenchFries)	Conjunction Introduction	From Steps 5 and 7
9	exists f [Serves(McD, f) & CanEat(c, f)]	Existential Generalization	From Step 8
10	Person(c) & Restaurant(McD) & exists f (Serves(McD, f) & CanEat(c, f))	Conjunction Introduction	From Steps 3, 1 (A5), and 9

Step	Derived Formula	Inference Rule Applied	Premises / Justification
11	CanEatAt(c, McD)	Modus Ponens with Axiom A3	From Step 10 and A3
12	Vegetarian(c) & CanEatAt(c, McD)	Conjunction Introduction	From Steps 2 and 11
13	exists x [Vegetarian(x) & CanEatAt(x, McD)]	Existential Generalization (x = c)	From Step 12 (Q.E.D.)

Resolution Refutation Proof for Assertion b

We negate the target assertion: NegGoal_b: ~exists x [Vegetarian(x) & CanEatAt(x, McD)], which converts into the CNF clause: C_negB: ~Vegetarian(x) v ~CanEatAt(x, McD). Resolving C_negB against the instantiated clauses from Axioms A3, A4, A7, and A8 through unification theta = { x / c } resolves all intermediate dietary permissions and produces the Empty Clause Box, completing the automated refutation proof.

3.8 Computational Complexity: Decidability of Definite Horn Clauses vs General FOL

A critical distinction in automated reasoning lies between full First-Order Logic and the Horn Clause subset. Full First-Order Logic is semi-decidable (Church-Turing Theorem): if an assertion alpha is entailed by KB, resolution refutation will terminate in finite time and derive Box; however, if alpha is NOT entailed, the theorem prover may loop infinitely searching for a proof.

To achieve computational tractability in commercial systems, knowledge engineers restrict axioms to Definite Horn Clauses (clauses containing exactly one positive literal): (p_1 & p_2 & ... & p_k) => q. Horn theories possess two remarkable properties:

1. Decidable Linear-Time Inference: Forward chaining with forward-directed unit resolution executes in O(N) linear time with respect to the number of knowledge base literals, using the Rete algorithm or graph indexing.
2. Existence of a Unique Minimal Model: Unlike general FOL where disjunctions introduce multiple alternative models, a definite Horn theory possesses a unique canonical minimal Herbrand model, representing the intersection of all satisfying models. This theoretical property ensures that database queries always return consistent, deterministic answers.

3.6 Real-World Sociological and Food-Science Complexities

While the First-Order Logic proof demonstrates that vegetarians can eat at McDonald's under the idealized premise that French Fries constitute vegetarian food, real-world computational ontology requires modeling nuanced food science variables. In 2001, a landmark legal case in the United States revealed that McDonald's French Fries contained natural beef flavoring added during par-frying at processing plants, rendering them strictly non-vegetarian in the US market, despite being 100% vegetarian in India. Furthermore, strict lacto-vegetarians and Jains reject food items subjected to deep-fryer cross-contamination with poultry or meat products. A high-fidelity neuro-symbolic knowledge graph must therefore incorporate non-monotonic modal qualifiers (e.g., RegionalRecipe(r, country) and CrossContamination(f)).

3.7 Backward Chaining SLD Derivation Trees & Closed World Assumption

While Section 3.3 and 3.4 formulated proofs using forward Natural Deduction and resolution refutation, modern logic programming engines (such as Prolog and Datalog) execute goal-directed deductive queries using SLD Resolution (Linear resolution with Selection function over Definite clauses). SLD resolution begins with the negated goal query ?- ~VegetarianRestaurant(McD) and searches backward through the knowledge base to find matching rule heads, recursively establishing subgoals.

Furthermore, in practical knowledge representation, systems must navigate the boundary between the Open World Assumption (OWA) and the Closed World Assumption (CWA):
- Open World Assumption (OWA): Employed by standard First-Order Logic and Semantic Web OWL-DL. If a fact cannot be proven from KB, it is considered unknown (undetermined), rather than false. In our proof that McDonald's is not vegetarian, we did NOT assume non-vegetarianism from lack of evidence; we derived it deductively from positive evidence (Big Mac contains meat).
- Closed World Assumption (CWA): Employed by relational databases and Prolog negation-as-failure. If a proposition cannot be derived, it is assumed to be false. Under CWA, if a restaurant menu does not explicitly mention meat, the system would falsely assume it serves no meat!

Table 3.5: Comparative Evaluation of Automated Deductive Inference Paradigms

Inference Strategy	Search Direction	Completeness Guarantee	Assumption Paradigm	Primary Implementation
Resolution Refutation	Bidirectional / Refutation	Refutation Complete for full FOL	Open World Assumption (OWA)	Automated Theorem Provers (Vampire, Prover9)
SLD Resolution	Backward Goal-Directed	Complete for Horn Clause subset	Closed World (Negation as Failure)	Prolog & Constraint Logic Programming
Forward Chaining	Data-Driven / Bottom-Up	Complete for First-Order Horn Theories	Open / Closed World	Production Rule Systems (CLIPS, Drools)
Tableau Method	Model-Building Refutation	Sound and Complete for FOL & DL	Open World Assumption (OWA)	Description Logic Reasoners (HermiT, Pellet)

4. Question 3: Temporal First-Order Logic & Reichenbachian Event Semantics

Question. Question 3 (10 Marks) (CO4) (BLT4): Give FOL translations for the following sentences that capture the temporal relationships between the events:
a. When Mary's flight departed, I ate lunch.
b. When Mary's flight departed, I had eaten lunch.

4.1 The Inadequacy of Atemporal First-Order Logic

A standard atemporal FOL representation of Sentences a and b would map both utterances to an identical conjunction of event predicates: exists e_1 exists e_2 [Depart(e_1, MaryFlight) & Eat(e_2, Speaker, Lunch)]. This completely erases the critical temporal distinction conveyed by the grammatical aspect: in Sentence a, the speaker ate lunch at the time of departure (or immediately following it), whereas in Sentence b, the speaker had already completed lunch prior to the departure. Capturing this distinction requires augmenting FOL with explicit temporal coordinates and event intervals.

4.2 Hans Reichenbach's Tripartite Temporal Architecture across the 12 English Tenses

To systematically model temporal relations, Hans Reichenbach (1947) posited that grammatical tense maps an Event time (E) to a Speech time (S) mediated by a Reference time (R). Table 4.1 details Reichenbach's formalization across all twelve English tense-aspect configurations, establishing the formal baseline for resolving Question 3.

Table 4.1: Exhaustive Reichenbachian Tense-Aspect Architecture for English

Tense & Aspect	Reichenbach Configuration	Linear Order	Exemplar Utterance	Semantic Description
Simple Present	E = R = S	E, R, S coincident	I eat lunch.	Event occurs concurrently with the moment of speech.
Simple Past	E = R < S	E, R before S	I ate lunch.	Event and reference vantage point precede speech time.

Tense & Aspect	Reichenbach Configuration	Linear Order	Exemplar Utterance	Semantic Description
Simple Future	S < R = E	S before R, E	I will eat lunch.	Event and reference vantage point follow speech time.
Past Perfect (Pluperfect)	E < R < S	E before R before S	I had eaten lunch.	Event is completed prior to a past reference anchor.
Present Perfect	E < R = S	E before R, S	I have eaten lunch.	Past event completed with current reference relevance.
Future Perfect	S < E < R	S before E before R	I will have eaten lunch.	Event will be completed prior to a future reference anchor.
Past Progressive	E contains R < S	R within E, before S	I was eating lunch.	Event was ongoing at the moment of past reference.
Present Progressive	E contains R = S	S, R within E	I am eating lunch.	Event is actively ongoing at the moment of speech.
Future Progressive	S < R within E	S before R within E	I will be eating lunch.	Event will be ongoing at a future reference point.
Past Perfect Progressive	E < R < S (durative)	E precedes R before S	I had been eating lunch.	Extended event ongoing prior to past reference anchor.
Present Perfect Progressive	E extends to R = S	E terminates at S	I have been eating lunch.	Extended event started in past and persists up to speech.
Future Perfect Progressive	S < E extends to R	S before E up to R	I will have been eating lunch.	Extended event persisting up to a future reference anchor.

4.3 James Allen's 13 Interval Temporal Relations

James Allen (1983) proved that exactly thirteen mutually exclusive, exhaustive binary topological relations exist between any two temporal intervals t_1 = [s_1, e_1] and t_2 = [s_2, e_2] (where s_i < e_i). Table 4.2 presents the complete ledger of Allen's 13 relations alongside their endpoint inequalities and ASCII timeline representations.

Table 4.2: James Allen's 13 Interval Temporal Relations with Endpoint Constraints

Relation Name	Symbol / Inverse	Endpoint Constraints	ASCII Timeline Representation
Before	p / pi (After)	e_1 < s_2	t_1: [---] t_2: [---]
Meets	m / mi (Met-By)	e_1 = s_2	t_1: [---] t_2: [---]
Overlaps	o / oi (Overlapped-By)	s_1 < s_2 < e_1 < e_2	t_1: [------] t_2: [------]
Starts	s / si (Started-By)	s_1 = s_2 & e_1 < e_2	t_1: [---] t_2: [------]
During	d / di (Contains)	s_2 < s_1 & e_1 < e_2	t_1: [---] t_2: [---------]
Finishes	f / fi (Finished-By)	s_2 < s_1 & e_1 = e_2	t_1: [---] t_2: [---------]
Equals	eq / eq	s_1 = s_2 & e_1 = e_2	t_1: [------] t_2: [------]

4.4 Formal Event Calculus Axiomatization (Shanahan / Kowalski)

In addition to interval algebra, Murray Shanahan (1999) and Robert Kowalski formalize continuous time reasoning through the Event Calculus. The Event Calculus operates over events e, time points t, and fluents f (time-varying properties of the world). The core axioms are:

1. HoldsAt(f, t_2) <= Happens(e, t_1) & Initiates(e, f) & (t_1 < t_2) & ~Clipped(t_1, f, t_2)
2. Clipped(t_1, f, t_2) <=> exists e exists t [Happens(e, t) & Terminates(e, f) & (t_1 < t < t_2)]

In Question 3, the fluent HasEaten(Speaker, Lunch) is initiated by the event Eat(Speaker, Lunch). In Sentence b, HasEaten holds at the exact timestamp t_depart of the flight departure, whereas in Sentence a, it does not hold at t_depart but is initiated during or after it.

4.5 Formalization of Sentence a: 'When Mary's flight departed, I ate lunch'

In Sentence a, both the subordinate clause ('When Mary's flight departed') and the main clause ('I ate lunch') are inflected in the Simple Past tense. Under Reichenbach's framework, both events share a common past reference time: E_1 = R_1 < S and E_2 = R_2 < S, with R_1 = R_2. Let TimeSpan(e) denote the temporal interval [t_{start}(e), t_{end}(e)] associated with event e, and let now denote Speech Time S.

Sentence a is formalized in Standard Temporal FOL as: exists e_1 exists e_2 [Depart(e_1, MaryFlight) & Eat(e_2, Speaker, Lunch) & TemporalOverlap(TimeSpan(e_2), TimeSpan(e_1)) & EndTime(TimeSpan(e_2)) < now]

In James Allen's Interval Logic, this corresponds to: exists e_1 exists e_2 [Depart(e_1, MaryFlight) & Eat(e_2, Speaker, Lunch) & (Overlaps(e_1, e_2) v Meets(e_1, e_2) v During(e_2, e_1)) & Before(TimeSpan(e_1), now)]

4.6 Formalization of Sentence b: 'When Mary's flight departed, I had eaten lunch'

In Sentence b, the subordinate clause is inflected in the Simple Past ('departed', setting reference event e_1), while the main clause is inflected in the Past Perfect ('had eaten', setting event e_2). Reichenbach's past perfect requires strict anteriority: E_2 < R_1 = E_1 < S. The lunch event e_2 was entirely completed prior to the departure of Mary's flight. Sentence b is formalized in Standard Temporal FOL as: exists e_1 exists e_2 [Depart(e_1, MaryFlight) & Eat(e_2, Speaker, Lunch) & EndTime(TimeSpan(e_2)) < StartTime(TimeSpan(e_1)) & EndTime(TimeSpan(e_1)) < now]

In James Allen's Interval Logic, this corresponds strictly and uniquely to the Before relation: exists e_1 exists e_2 [Depart(e_1, MaryFlight) & Eat(e_2, Speaker, Lunch) & Before(TimeSpan(e_2), TimeSpan(e_1)) & Before(TimeSpan(e_1), now)]

4.7 Comparative Synthesis of Temporal Representations

Table 4.3 synthesizes the linguistic aspect, Reichenbachian coordinates, Allen relations, and Event Calculus fluent states for Sentences a and b.

Table 4.3: Comparative Synthesis of Temporal Formalisms for Question 3

Sentence	Grammatical Aspect	Reichenbach Formulation	Allen Interval Relation	Event Calculus State at Departure

Sentence	Grammatical Aspect	Reichenbach Formulation	Allen Interval Relation	Event Calculus State at Departure
a. When Mary's flight departed, I ate lunch	Simple Past + Simple Past	E_1 = R_1 < S E_2 = R_2 < S R_1 = R_2	Overlaps(e_1, e_2) v Meets(e_1, e_2) v During(e_2, e_1)	~HoldsAt(HasEaten(Speaker, Lunch), StartTime(e_1))
b. When Mary's flight departed, I had eaten lunch	Simple Past + Past Perfect	E_2 < R_1 = E_1 < S	Before(TimeSpan(e_2), TimeSpan(e_1))	HoldsAt(HasEaten(Speaker, Lunch), StartTime(e_1))

4.8 Discourse Representation Theory (DRT) and Temporal Anaphora

To model how temporal subordinate clauses anchor subsequent discourse across multiple sentences, Hans Kamp (1981) developed Discourse Representation Theory (DRT). A Discourse Representation Structure (DRS) consists of two components:
1. Universe of Discourse U: A set of discourse referents (variables representing individuals, events, and temporal intervals).
2. Conditions C: A set of descriptive and relational conditions constraining those referents.

In Sentence 3a ('When Mary's flight departed, I ate lunch'), the subordinate temporal clause introduces an event referent e_1 and a temporal interval t_1 into the universe. The main clause introduces event e_2 and interval t_2. The temporal connective 'when' generates the DRS condition t_2 subseteq t_1 (temporal overlap). In contrast, in Sentence 3b ('When Mary's flight departed, I had eaten lunch'), the past perfect introduces an anterior reference point, generating the condition t_2 < t_1 (strict precedence).

Table 4.4: Discourse Representation Structure (DRS) Formal Specifications for Sentences 3a and 3b

Discourse Representation Parameter	Sentence 3a: 'I ate lunch'	Sentence 3b: 'I had eaten lunch'
Discourse Referents (Universe U)	{ x, y, e_1, e_2, t_1, t_2, n }	{ x, y, e_1, e_2, t_1, t_2, t_ref, n }
Entity Conditions	MaryFlight(x), Speaker(y), Lunch(z)	MaryFlight(x), Speaker(y), Lunch(z)
Event Conditions	Depart(e_1, x), Eat(e_2, y, z)	Depart(e_1, x), Eat(e_2, y, z)

Discourse Representation Parameter	Sentence 3a: 'I ate lunch'	Sentence 3b: 'I had eaten lunch'
Temporal Span Constraints	TimeSpan(e_1) = t_1, TimeSpan(e_2) = t_2	TimeSpan(e_1) = t_1, TimeSpan(e_2) = t_2
Aspectual Relational Conditions	t_2 Overlaps t_1, t_1 < n	t_2 < t_ref, t_ref = t_1, t_1 < n
Discourse Anaphoric Resolution	Lunch event is temporally anchored during departure	Lunch event is completely anterior to departure anchor

4.9 Temporal Constraint Satisfaction in Spoken Dialogue Management

In practical NLP applications such as the Air Travel Information System (ATIS), temporal representations are not merely theoretical artifacts; they function as active constraints in Temporal Constraint Satisfaction Problems (TCSPs). A flight reservation dialogue engine must dynamically evaluate whether a user's requested itinerary is temporally feasible. Consider an ATIS query: 'I want a flight from Atlanta arriving in Dallas before 3 p.m., with at least a 45-minute layover.' The dialogue manager maps this request to a Temporal Constraint Network (TCN) where nodes represent event endpoints and directed edges represent allowable duration intervals [d_min, d_max]. By applying the Floyd-Warshall all-pairs shortest path algorithm over the Distance Graph, the dialogue manager computes the minimal network in O(V^3) time, immediately detecting scheduling conflicts or identifying valid flight connections without human intervention.

5. Question 4: Critique and Repair of the Defective Definition

Question. Question 4 (10 Marks) (CO4) (BLT4): The following rule, which we gave as a translation, is not a reasonable definition of what it means to be a vegetarian restaurant: forall x VegetarianRestaurant(x) => Serves(x, VegetarianFood)

5.1 The Four Philosophical and Mathematical Fallacies of the Rule

The proposed rule forall x [VegetarianRestaurant(x) => Serves(x, VegetarianFood)] suffers from four profound theoretical, logical, and empirical fallacies that completely disqualify it as a valid definitional formula in computational semantics:

Fallacy 1: The Material Implication Paradox (Vacuous Truth)

In classical logic, a material implication p => q is logically true whenever the antecedent p is false (the principle of ex falso quodlibet). Because the quantifier forall x ranges over all entities in the universe, let x be an entity that is not a vegetarian restaurant (e.g., a boulder, a submarine, the planet Jupiter, or a traditional steakhouse). For every such entity, VegetarianRestaurant(x) is False. Consequently, the implication False => Serves(x, VegetarianFood) evaluates to True! The rule therefore asserts that everything in the universe satisfies the formula, failing to impose any semantic constraint whatsoever on non-vegetarian entities.

Fallacy 2: Insufficiency as a Definition (Directionality Defect)

In formal logic and lexical semantics, a definition of a concept must provide both Necessary and Sufficient conditions (an 'if and only if' bi-conditional, phi <=> psi). The given rule provides only a one-way conditional (Necessary condition): it states that IF x is a vegetarian restaurant, then x serves vegetarian food. It completely fails to state what conditions SUFFICE to classify an unknown establishment as a vegetarian restaurant. An automated classifier or theorem prover using this rule can never deduce VegetarianRestaurant(x) from menu evidence.

Fallacy 3: The Steakhouse Counterexample (Lack of Exclusivity)

The most devastating empirical flaw is that the consequent Serves(x, VegetarianFood) is satisfied by virtually every commercial restaurant on Earth. Consider an authentic Brazilian Steakhouse or a Texas BBQ Smokehouse: in addition to serving beef and pork, it invariably

serves steamed rice, baked potatoes, and garden salads. Because salad is vegetarian food, Serves(TexasBBQ, Salad) is True! If the rule were treated as a characterization of vegetarian restaurants, it would either fail to distinguish a vegetarian restaurant from a steakhouse, or if inverted, would absurdly classify the steakhouse as a vegetarian restaurant.

Fallacy 4: Lack of Sortal Filtering (Untyped Universal Quantifier)

The quantifier forall x is untyped, lacking a sortal domain restriction. A proper ontological rule must explicitly restrict x to the category Restaurant(x) and must assert that the restaurant is an active food-serving establishment (exists y Serves(x, y)).

5.2 Model-Theoretic Counter-Model Construction (Tarskian Truth Evaluation)

To mathematically prove the invalidity of the rule as a definition, we construct two formal Tarskian counter-models.

Counter-Model M_1: The Omnivorous Steakhouse Model

Let model M_1 = (D, I) have Domain of Discourse D = { r_steak, f_beef, f_salad }. The interpretation function I is defined as follows:

- I(Restaurant) = { r_steak }
- I(VegetarianRestaurant) = empty set (r_steak is an authentic steakhouse, hence NOT vegetarian)
- I(Meat) = { f_beef }
- I(VegetarianFood) = { f_salad }
- I(Serves) = { (r_steak, f_beef), (r_steak, f_salad) }

Evaluation: In model M_1, for entity r_steak, the antecedent VegetarianRestaurant(r_steak) is False. Therefore, the material implication VegetarianRestaurant(r_steak) => Serves(r_steak, VegetarianFood) evaluates to True! Furthermore, if one attempted to treat the rule as a definition by inverting it (Serves(x, VegetarianFood) => VegetarianRestaurant(x)), the formula would evaluate to False because Serves(r_steak, f_salad) is True yet VegetarianRestaurant(r_steak) is False. This formally proves that material implication cannot model concept definitions in natural language semantics.

Counter-Model M_2: The Defunct / Empty Bistro Model

Let model M_2 = (D, I) have Domain D = { r_empty }, where r_empty is a newly registered vegetarian restaurant that has not yet opened its kitchen and currently serves no food items: I(Serves) = empty set. Under a naive definition requiring exists f (Serves(r, f) & VegFood(f)), r_empty would be disqualified. This proves that an industrial definition must distinguish between the business category of an establishment and its active daily menu offerings.

5.3 Description Logic (ALC / OWL-DL) Formalization

In modern semantic web knowledge bases, concept definitions are formalized in Description Logics (Baader et al., 2003) to ensure computational decidability. In the Description Logic ALC, the concept VegetarianRestaurant is formally defined using concept inclusion and value restriction axioms: VegetarianRestaurant equiv Restaurant sqcap forall serves.VegetarianFood sqcap exists serves.VegetarianFood

This DL axiom guarantees that: (1) every vegetarian restaurant is a restaurant; (2) every food item served belongs to the concept VegetarianFood; and (3) there exists at least one served food item. Furthermore, disjointness is asserted: Meat sqcap VegetarianFood equiv bottom.

5.4 Three Robust First-Order Logic Repair Formulations

We engineer three complete, mathematically robust First-Order Logic repair formulations:

Repair Formulation 1: Sortal Bi-Conditional with Universal Exclusivity

This formulation defines a vegetarian restaurant as an establishment that is a restaurant, serves at least one food item, and where every food item served is vegetarian:

forall x [VegetarianRestaurant(x) <=> (Restaurant(x) &
forall y (Serves(x, y) => VegetarianFood(y)) &
exists y Serves(x, y))]

Repair Formulation 2: Negative Exclusivity (Strict Meat Prohibition)

Focusing on dietary prohibition, a vegetarian restaurant is defined as an establishment that serves food but serves absolutely zero meat:

forall x [VegetarianRestaurant(x) <=> (Restaurant(x) &
~exists y (Serves(x, y) & Meat(y)) &
exists y Serves(x, y))]

Repair Formulation 3: Neo-Davidsonian Menu-Level Event Ontology

In enterprise knowledge graphs, restaurants are linked to official menus m containing dishes d. A restaurant is vegetarian if and only if all dishes on its active menu conform to the vegetarian dietary restriction:

forall x [VegetarianRestaurant(x) <=> (Restaurant(x) & forall m (HasMenu(x, m) => forall d (ContainsDish(m, d) => VegetarianDish(d))) & exists m exists d (HasMenu(x, m) & ContainsDish(m, d)))]

5.5 Comparative Evaluation of Proposed Repairs

Table 5.1 compares the original defective rule against our three proposed repairs across six formal semantic criteria.

Table 5.1: Comparative Theoretical Evaluation of Definitional Repairs

Formulation	Necessary?	Sufficient?	Sortal Filter?	Exclusivity?	Steakhouse Safe?
Original Rule: forall x (VegRest(x) => Serves(x, VegFood))	Yes	No (1-Way)	No (Untyped)	No (Allows Meat)	No (Fails Catastrophically)
Repair 1: Bi-conditional Universal Exclusivity	Yes	Yes (<=>)	Yes (Restaurant(x))	Yes (All items veg)	Yes (100% Robust)
Repair 2: Negative Meat Prohibition	Yes	Yes (<=>)	Yes (Restaurant(x))	Yes (Zero meat)	Yes (100% Robust)
Repair 3: Neo-Davidsonian Menu Ontology	Yes	Yes (<=>)	Yes (Restaurant(x))	Yes (All dishes veg)	Yes (Enterprise Grade)

5.6 Non-Monotonic Reasoning, Default Logic, and Circumscription

In classical First-Order Logic, reasoning is strictly monotonic: if KB |= alpha, then for any additional axiom gamma, KB union { gamma } |= alpha. New information can never invalidate previously derived theorems. However, common-sense reasoning regarding restaurant categories is naturally defeasible (non-monotonic). Consider the default assumption: 'Restaurants typically serve food that aligns with their primary branding, unless an explicit exception is known.'

To formalize such defeasible dietary rules without the paradoxes of material implication, Raymond Reiter (1980) formulated Default Logic. A default rule has the schema: alpha : beta / gamma (If alpha is known, and it is consistent to assume beta, then infer gamma). Furthermore, John McCarthy (1980) introduced Circumscription, which minimizes the extension of an abnormal predicate Ab(x). In our restaurant domain, this is formalized as: forall x [(Restaurant(x) & ~Ab_1(x)) => (VegetarianRestaurant(x) <=> forall y (Serves(x, y) => VegetarianFood(y)))]

By circumscribing Ab_1, anomalous edge cases (such as pop-up catering events or experimental hybrid menus) are isolated without corrupting the core ontological definition.

5.7 Formal Concept Analysis (FCA) and Ontological Galois Lattices

To provide a rigorous mathematical foundation for dietary categorization, we formalize restaurant classification using Formal Concept Analysis (FCA) (Ganter & Wille, 1999). A formal context is a triple K = (G, M, I), where G is a set of objects (restaurants), M is a set of attributes (dietary offerings), and I subseteq G x M is a binary incidence relation indicating that object g has attribute m.

Let G = { McDonald's, Saravana Bhavan, Texas Steakhouse, Loving Hut (Vegan) } and M = { ServesMeat, ServesDairy, ServesSalad, ExclusivelyVegetarian, CertifiedVegan }. Table 5.2 defines the formal context matrix.

Table 5.2: Formal Context Cross-Table for Dietary Restaurant Taxonomy

Restaurant Object (g)	ServesMeat	ServesDairy	ServesSalad	ExclusivelyVegetarian	CertifiedVegan
McDonald's	X	X	X		
Saravana Bhavan (Pure Veg)		X	X	X	
Texas Steakhouse	X		X		
Loving Hut (100% Vegan)			X	X	X

From this formal context, the Galois connection derives the Concept Lattice. A formal concept is a pair (A, B) where A subseteq G is the extent (objects) and B subseteq M is the intent (attributes) such that A' = B and B' = A. The concept 'Vegetarian Restaurant' has extent

{ Saravana Bhavan, Loving Hut } and intent { ServesSalad, ExclusivelyVegetarian }. Crucially, McDonald's shares the attribute 'ServesSalad' with vegetarian restaurants, but is strictly excluded from the concept 'Vegetarian Restaurant' because its attribute set contains 'ServesMeat', which violates the exclusivity requirement B' = A. This lattice-theoretic proof confirms that the defective definition failed because it matched on shared sub-attributes rather than maximal closed concept intents.

6. Synthesis, Neuro-Symbolic Integration, and Modern Horizons

6.1 Master Synthesis of Empirical Findings

Table 6.1 synthesizes the core mathematical proofs, logical formalisms, and computational results established across all four questions of Assignment 4.

Table 6.1: Master Synthesis of Empirical Findings for Assignment 4

Assignment Component	Methodological Focus	Key Theoretical & Empirical Result	Computational Takeaway
Question 1: FOL Translations	Quantifier Scope & Equivalence	Formulated 3 equivalent FOL translations for Sentence a and 2 for Sentence b; proved equivalence via De Morgan's laws	Wide-scope universal with implication is isomorphic to negated existential conjunction.
Question 2: Knowledge Base & Proofs	Automated Resolution Refutation	Engineered 8-axiom KB; derived Box via binary resolution refutation for McD non-veg and vegetarian dining permissibility	Generalized ontological axioms enable automated deductive theorem proving without hand-crafting target facts.
Question 3: Temporal Semantics	Reichenbach (S, R, E) & Allen Intervals	Formalized Simple Past (E, R < S) vs Past Perfect (E < R < S); mapped 'when' to During/Meets vs Before	Grammatical aspect alters temporal interval topology, requiring explicit interval coordinates in FOL.
Question 4: Definitional Repairs	Philosophical Logic & Fallacy Analysis	Exposed 4 fatal fallacies in material implication; engineered 3 bi-conditional repairs with sortal filtering	Definitional assertions require bi-conditionals (<=>) and universal exclusivity to prevent vacuous truth.

6.2 The Rise of Neuro-Symbolic AI and Semantic Web Technologies

In contemporary artificial intelligence, pure symbolic First-Order Logic and pure deep neural networks represent complementary paradigms. While Large Language Models (LLMs) excel at statistical language generation, common sense reasoning, and soft pattern matching, they suffer from logical hallucinations, lack formal verifiability, and struggle with strict deductive chains. Neuro-Symbolic AI integrates the perceptual strength of neural representations with the rigorous, explainable proof mechanisms of First-Order Logic. In Semantic Web engineering, Description Logics (a decidable fragment of FOL) underpin the Web Ontology

Language (OWL-DL), enabling enterprise knowledge graphs to execute automated consistency checks, verify regulatory compliance, and power explainable question-answering.

6.3 Continuous Automated Regression Testing for Formal Knowledge Bases

In modern enterprise knowledge engineering, maintaining logical consistency across hundreds of thousands of First-Order Logic axioms requires continuous integration and verification frameworks. Just as software engineering mandates unit testing suites, ontology engineering employs automated regression testing using reasoners such as Vampire, Prover9, and HermiT.

Whenever a domain ontology (such as SNOMED-CT for medical terminology or Schema.org for commercial entities) is updated with new class definitions, automated pipelines verify three formal invariants:

1. Concept Satisfiability: Verifying that no non-bottom concept C evaluates to the empty set (C equiv bottom). For instance, an erroneous axiom that asserts a vegetarian restaurant must serve meat would render VegetarianRestaurant unsatisfiable.
2. Knowledge Base Consistency: Checking that the conjunction of all axioms admits at least one model (KB != false).
3. Entailment Invariance: Confirming that all established core theorems (such as Assertion 2a: ~VegetarianRestaurant(McD)) remain strictly derivable without regression.

Appendix A: Complete Python Implementation of Resolution Refutation Prover

Listing A.1 provides a complete, object-oriented Python implementation of an automated Propositional and Clausal Resolution Theorem Prover. The script verifies the unsatisfiability of the negated goal for Question 2, automatically deriving the empty clause Box.

```
"""
resolution_prover.py - Automated Clausal Resolution Refutation Theorem Prover
Course: DSA03 - Natural Language Processing | SIMATS School of Engineering
Student: Appala Manohar | Register Number: 192425360 | Year: 2026
"""


class Literal:
    """Represents a signed atomic literal (positive or negative)."""
    def __init__(self, name, negated=False):
        self.name = name
        self.negated = negated


    def complement(self):
        return Literal(self.name, not self.negated)


    def __eq__(self, other):
        return self.name == other.name and self.negated == other.negated


    def __hash__(self):
        return hash((self.name, self.negated))


    def __repr__(self):
        return f"~{self.name}" if self.negated else self.name



class Clause:
    """Represents a disjunctive clause: L1 v L2 v ... v Lk."""
    def __init__(self, literals):
        self.literals = frozenset(literals)


    def is_empty(self):
        return len(self.literals) == 0


    def __eq__(self, other):
        return self.literals == other.literals


    def __hash__(self):
        return hash(self.literals)


    def __repr__(self):
        if self.is_empty():
            return "[] (EMPTY_CLAUSE_BOX)"
        return " v ".join(str(lit) for lit in self.literals)



def resolve(c1, c2):
    """Computes all possible binary resolvents between two clauses."""
    resolvents = []
    for lit1 in c1.literals:
        comp = lit1.complement()
        if comp in c2.literals:
            new_lits = (c1.literals - {lit1}) | (c2.literals - {comp})
            resolvents.append(Clause(new_lits))
    return resolvents
```

```
def prove_by_resolution(clauses):
    """Executes systematic resolution refutation to derive the empty clause."""
    clause_set = set(clauses)
    step = 1


    print("Initial Knowledge Base Clauses + Negated Goal:")
    for idx, c in enumerate(clauses):
        print(f"  C_{idx+1}: {c}")


    while True:
        new_clauses = set()
        clause_list = list(clause_set)


        for i in range(len(clause_list)):
            for j in range(i + 1, len(clause_list)):
                c1 = clause_list[i]
                c2 = clause_list[j]
                resolvents = resolve(c1, c2)


                for res in resolvents:
                    if res.is_empty():
                        print(f"\nStep {step}: Resolved C_{i+1} and C_{j+1} -> Derived [] (CONTRADICTION BOX)")
                        return True
                    if res not in clause_set:
                        new_clauses.add(res)
                        print(f"Step {step}: Resolved ({c1}) and ({c2}) -> Derived: ({res})")
                        step += 1


        if new_clauses.issubset(clause_set):
            return False


        clause_set.update(new_clauses)



if __name__ == "__main__":
    C1 = Clause([Literal("Meat", negated=True), Literal("VegFood", negated=True)])
    C2 = Clause([Literal("VegRest", negated=True), Literal("VegFood", negated=False)])
    C3 = Clause([Literal("Meat", negated=False)])
    C4_NegGoal = Clause([Literal("VegRest", negated=False)])


    print("--- Executing Resolution Refutation for Assertion 2a ---")
    contradiction_found = prove_by_resolution([C1, C2, C3, C4_NegGoal])
    print(f"\nContradiction Established: {contradiction_found}")
    print("Assertion 2a (~VegRest) Formally Proved via Resolution Refutation!")
```

Appendix B: Python Implementation of Allen's Interval Temporal Reasoner

Listing B.1 provides a Python module implementing James Allen's Interval Algebra relations and validating the temporal entailments between departure and dining intervals for Question 3.

```
"""
allen_temporal_reasoner.py - James Allen's 13 Interval Temporal Relations
Course: DSA03 - Natural Language Processing | SIMATS School of Engineering
Student: Appala Manohar | Register Number: 192425360 | Year: 2026
"""


class TimeInterval:
    """Represents a closed temporal interval [start, end]."""
    def __init__(self, name, start, end):
        assert start < end, "Start time must strictly precede end time"
        self.name = name
        self.start = start
        self.end = end


    def relation_to(self, other):
        """Determines the exact Allen interval relation between self and other."""
        if self.end < other.start:
            return "BEFORE"
        elif self.end == other.start:
            return "MEETS"
        elif self.start < other.start and self.end > other.start and self.end < other.end:
            return "OVERLAPS"
        elif self.start == other.start and self.end < other.end:
            return "STARTS"
        elif self.start > other.start and self.end < other.end:
            return "DURING"
        elif self.start > other.start and self.end == other.end:
            return "FINISHES"
        elif self.start == other.start and self.end == other.end:
            return "EQUALS"
        elif other.end < self.start:
            return "AFTER"
        elif other.end == self.start:
            return "MET_BY"
        elif other.start < self.start and other.end > self.start and other.end < self.end:
            return "OVERLAPPED_BY"
        elif other.start == self.start and other.end < self.end:
            return "STARTED_BY"
        elif other.start > self.start and other.end < self.end:
            return "CONTAINS"
        elif other.start > self.start and other.end == self.end:
            return "FINISHED_BY"
        return "UNKNOWN"


if __name__ == "__main__":
    t_flight_a = TimeInterval("FlightDeparture_A", 10.0, 10.5)
    t_lunch_a = TimeInterval("Lunch_A", 10.1, 10.4)
    print(f"Sentence 3a Relation: Lunch is {t_lunch_a.relation_to(t_flight_a)} Flight Departure")


    t_lunch_b = TimeInterval("Lunch_B", 8.5, 9.5)
    t_flight_b = TimeInterval("FlightDeparture_B", 10.0, 10.5)
    print(f"Sentence 3b Relation: Lunch is {t_lunch_b.relation_to(t_flight_b)} Flight Departure")
```

Appendix C: Comprehensive Academic Glossary of Logic & Semantics

Table C.1 provides rigorous, academic definitions for thirty foundational concepts spanning First-Order Logic, computational semantics, automated theorem proving, and temporal reasoning.

Table C.1: Comprehensive Academic Glossary of Logic & Semantics

No.	Technical Term	Formal Definition & Theoretical Significance
1	First-Order Logic (FOL)	A formal mathematical language with predicates, functions, constants, variables, and universal and existential quantifiers.
2	Model Theory	The semantic framework formalizing the truth of logical sentences relative to an interpretation over a domain of discourse.
3	Domain of Discourse (D)	The non-empty set of all individual objects, entities, and events existing within a model.
4	Interpretation Function (I)	A mapping assigning domain entities to constants, relations to predicates, and operations to functions.
5	Principle of Compositionality	The principle that the meaning of a complex expression is a function of the meanings of its parts.
6	Lambda Calculus	A formal mathematical system for function definition, application, and recursion developed by Alonzo Church.
7	Beta-Reduction	The operational substitution rule in lambda calculus replacing bound variables with argument values.
8	Neo-Davidsonian Semantics	An event-centric semantic framework reifying events as first-class domain entities linked to thematic roles.
9	Thematic Roles	Semantic relations (Agent, Patient, Theme, Goal) characterizing the participatory role of an entity in an event.
10	Conjunctive Normal Form (CNF)	A normalized logical formula structured as a conjunction of disjunctive clauses.
11	Skolemization	The deterministic transformation replacing existentially quantified variables with Skolem constants or functions.
12	Most General Unifier (MGU)	The minimal variable substitution that makes two distinct logical atomic expressions syntactically identical.
13	Resolution Refutation	A refutation-complete inference algorithm proving KB |= alpha by showing KB & ~alpha yields the empty clause Box.
14	Empty Clause (Box)	A clause containing zero literals, representing an absolute logical contradiction.
15	Material Implication	A logical connective (p => q) defined as ~p v q, which evaluates to true whenever the antecedent p is false.

No.	Technical Term	Formal Definition & Theoretical Significance
16	Ex Falso Quodlibet	The logical principle that from a false premise, any arbitrary proposition can be derived (vacuous truth).
17	Bi-conditional (<=>)	A logical connective asserting mutual necessary and sufficient conditionality: (p => q) & (q => p).
18	Necessary Condition	A condition that must be true for a statement to hold; failure of the condition guarantees failure of the statement.
19	Sufficient Condition	A condition whose truth guarantees the truth of the statement.
20	Sortal Filtering	The ontological practice of restricting variable quantification to a specific conceptual type or sort.
21	Speech Time (S)	In Reichenbach's temporal logic, the objective point in time at which an utterance is produced.
22	Event Time (E)	In Reichenbach's temporal logic, the temporal span during which the denoted event takes place.
23	Reference Time (R)	In Reichenbach's temporal logic, the subjective temporal perspective from which the event is considered.
24	Simple Past	A tense configuration where Event time and Reference time coincide prior to Speech time: E, R < S.
25	Past Perfect	A grammatical aspect where Event time strictly precedes Reference time prior to Speech time: E < R < S.
26	Allen Interval Algebra	A formal calculus defining thirteen mutually exclusive binary temporal relations between extended intervals.
27	Event Calculus	A logical formalism for reasoning about events, actions, and their persistent effects over time.
28	Knowledge Base (KB)	A formalized repository of axioms, rules, and facts representing domain knowledge.
29	Logical Entailment (|=)	The relation where a conclusion sentence is true in every model where the premises are true.
30	Neuro-Symbolic AI	An artificial intelligence paradigm synthesizing deep neural networks with formal symbolic reasoning.

Appendix D: Formal Academic References (APA 7th Edition)

The following academic bibliography lists the foundational literature, authoritative textbooks, and seminal papers informing the theoretical proofs, knowledge representations, and temporal formalisms in this treatise:

Allen, J. F. (1983). Maintaining knowledge about temporal intervals. Communications of the ACM, 26(11), 832-843.

Baader, F., Calvanese, D., McGuinness, D. L., Nardi, D., & Patel-Schneider, P. F. (Eds.). (2003). The Description Logic Handbook: Theory, Implementation, and Applications. Cambridge University Press.

Barwise, J., & Cooper, R. (1981). Generalized quantifiers and natural language. Linguistics and Philosophy, 4(2), 159-219.

Blackburn, P., & Bos, J. (2005). Representation and Inference for Natural Language: A First Course in Computational Semantics. CSLI Publications.

Church, A. (1940). A formulation of the simple theory of types. The Journal of Symbolic Logic, 5(2), 56-68.

Davidson, D. (1967). The logical form of action sentences. In N. Rescher (Ed.), The Logic of Decision and Action (pp. 81-95). University of Pittsburgh Press.

Dowty, D. R. (1991). Thematic proto-roles and argument selection. Language, 67(3), 547-619.

Frege, G. (1892). Über Sinn und Bedeutung [On sense and reference]. Zeitschrift für Philosophie und philosophische Kritik, 100, 25-50.

Genesereth, M. R., & Nilsson, N. J. (1987). Logical Foundations of Artificial Intelligence. Morgan Kaufmann.

Jurafsky, D., & Martin, J. H. (2024). Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition (3rd ed. draft). Pearson Prentice Hall.

Kowalski, R. (1979). Logic for Problem Solving. North Holland.

Manning, C. D., & Schütze, H. (1999). Foundations of Statistical Natural Language Processing. MIT Press.

Montague, R. (1974). Formal Philosophy: Selected Papers of Richard Montague (R. H. Thomason, Ed.). Yale University Press.

Parsons, T. (1990). Events in the Semantics of English: A Study in Subatomic Semantics. MIT Press.

Reichenbach, H. (1947). Elements of Symbolic Logic. Macmillan.

Robinson, J. A. (1965). A machine-oriented logic based on the resolution principle. Journal of the ACM, 12(1), 23-41.

Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

Shanahan, M. (1999). The Event Calculus explained. In M. J. Wooldridge & M. Veloso (Eds.), Artificial Intelligence Today (pp. 409-430). Springer.

Tarski, A. (1944). The semantic conception of truth and the foundations of semantics. Philosophy and Phenomenological Research, 4(3), 341-376.

Appendix E: Python Event Calculus Simulation Engine

Listing E.1 provides an executable Python simulation of the discrete Event Calculus. The simulator models the dynamic initiation and clipping of fluents across discrete time ticks, demonstrating that HasEaten(Speaker, Lunch) holds during the departure timestamp in Sentence 3b but is not initiated until after departure in Sentence 3a.

```
"""
event_calculus_sim.py - Discrete Event Calculus Simulation Engine
Course: DSA03 - Natural Language Processing | SIMATS School of Engineering
Student: Appala Manohar | Register Number: 192425360 | Year: 2026
"""


class EventCalculus:
    def __init__(self):
        self.events = []      # List of (event_name, timestamp)
        self.initiates = []   # List of (event_name, fluent)
        self.terminates = []  # List of (event_name, fluent)


    def add_event(self, event_name, t):
        self.events.append((event_name, t))
        self.events.sort(key=lambda x: x[1])


    def add_initiates(self, event_name, fluent):
        self.initiates.append((event_name, fluent))


    def add_terminates(self, event_name, fluent):
        self.terminates.append((event_name, fluent))


    def holds_at(self, fluent, query_time):
        """Axiom 1: HoldsAt(f, t) <= Happens(e, t_start) & Initiates(e, f) & ~Clipped."""
        # Find latest event prior to query_time that initiates fluent
        init_times = [t for (e, t) in self.events if (e, fluent) in self.initiates and t < query_time]
        if not init_times:
            return False
        latest_init = max(init_times)


        # Check if clipped (terminated between latest_init and query_time)
        term_times = [t for (e, t) in self.events if (e, fluent) in self.terminates and latest_init < t <
query_time]
        return len(term_times) == 0


if __name__ == "__main__":
    # Simulation for Sentence 3b: When Mary's flight departed (t=10), I had eaten lunch (eat at t=7)
    ec_3b = EventCalculus()
    ec_3b.add_initiates("Eat_Lunch", "HasEaten")
    ec_3b.add_event("Eat_Lunch", 7)
    ec_3b.add_event("Depart_Flight", 10)


    print("Sentence 3b: Departure at t=10, Lunch at t=7")
    print(f"  HoldsAt(HasEaten, t=10): {ec_3b.holds_at('HasEaten', 10)}")  # True!


    # Simulation for Sentence 3a: When Mary's flight departed (t=10), I ate lunch (eat at t=10.5)
    ec_3a = EventCalculus()
    ec_3a.add_initiates("Eat_Lunch", "HasEaten")
    ec_3a.add_event("Depart_Flight", 10)
    ec_3a.add_event("Eat_Lunch", 11)


    print("\nSentence 3a: Departure at t=10, Lunch at t=11")
    print(f"  HoldsAt(HasEaten, t=10): {ec_3a.holds_at('HasEaten', 10)}")  # False!
```

Appendix F: Python Implementation of Discourse Representation Structure (DRS) Parser

Listing F.1 provides an executable Python module modeling Discourse Representation Structures (DRS) for temporal anaphora. The module dynamically instantiates discourse referents, verifies aspectual constraints, and validates temporal consistency.

```
"""
drs_temporal_parser.py - Discourse Representation Structure (DRS) for Temporal Anaphora
Course: DSA03 - Natural Language Processing | SIMATS School of Engineering
Student: Appala Manohar | Register Number: 192425360 | Year: 2026
"""


class DRS:
    """Represents a Kamp Discourse Representation Structure."""
    def __init__(self, name):
        self.name = name
        self.universe = set()
        self.conditions = []


    def add_referent(self, var):
        self.universe.add(var)


    def add_condition(self, cond):
        self.conditions.append(cond)


    def display(self):
        print(f"=== DRS Box: {self.name} ===")
        print(f"Universe: {sorted(list(self.universe))}")
        print("Conditions:")
        for c in self.conditions:
            print(f"  * {c}")
        print("================================
")


if __name__ == "__main__":
    # Sentence 3a DRS
    drs_3a = DRS("Sentence_3a_SimplePast")
    for r in ["x", "y", "e_1", "e_2", "t_1", "t_2", "now"]:
        drs_3a.add_referent(r)
    drs_3a.add_condition("MaryFlight(x)")
    drs_3a.add_condition("Depart(e_1, x)")
    drs_3a.add_condition("Eat(e_2, Speaker, Lunch)")
    drs_3a.add_condition("TimeSpan(e_1) = t_1")
    drs_3a.add_condition("TimeSpan(e_2) = t_2")
    drs_3a.add_condition("Overlaps(t_2, t_1) or Meets(t_1, t_2)")
    drs_3a.add_condition("t_1 < now")
    drs_3a.display()


    # Sentence 3b DRS
    drs_3b = DRS("Sentence_3b_PastPerfect")
    for r in ["x", "y", "e_1", "e_2", "t_1", "t_2", "t_ref", "now"]:
        drs_3b.add_referent(r)
    drs_3b.add_condition("MaryFlight(x)")
    drs_3b.add_condition("Depart(e_1, x)")
    drs_3b.add_condition("Eat(e_2, Speaker, Lunch)")
    drs_3b.add_condition("TimeSpan(e_1) = t_1")
    drs_3b.add_condition("TimeSpan(e_2) = t_2")
    drs_3b.add_condition("t_ref = t_1")
    drs_3b.add_condition("Before(t_2, t_ref)")
    drs_3b.add_condition("t_1 < now")
    drs_3b.display()
```